

Take-Home Assessment — Task 2

Physics Experiments Suite · Three.js · 2–3 days

ROLE LEVEL	Mid frontend / 3D developer
TIMEBOX	2–3 days
CORE STACK	Three.js, JavaScript or TypeScript, any bundler (Vite recommended)
THEME	A mini virtual physics lab platform with three swappable experiments

Goal

Build a small physics-lab **platform**: three classic experiments running on one shared framework. The architecture is the point — the experiments must be interchangeable modules sharing a common interface, UI, and graphing system, and the physics must be numerically correct.

The platform must also be **extensible by an AI agent**. After you submit, we will take a specification for a physics experiment you have never seen, hand it to an AI agent together with your repository, and give it no hints from us — no walkthrough, no pointers, nothing beyond what is committed. We then score how cleanly that experiment drops into your platform. The fewer core and framework files it has to touch, the better you score.

Required scope

1 Shared framework (mandatory architecture)

- A common `Experiment` interface: `setup`, `update(dt)`, `reset`, `getMeasurements`, and a declared parameter schema.
- A UI panel that **auto-generates sliders/inputs from each experiment's parameter schema** — no per-experiment hand-built UI.
- A shared graphing module (2D overlay is fine) that plots any recorded quantity vs. time, with **CSV export** of the recorded data.
- A **fixed-timestep simulation loop decoupled from the render loop** (accumulator pattern). This will be checked: physics results must not change with frame rate.

- An experiment switcher in the UI that cleanly disposes one experiment and loads another (watch for Three.js resource leaks — dispose geometries/materials).
- The framework contract must be **discoverable and extensible by an AI agent working alone**. Assume a fourth experiment will be added later by an agent that has only your repository to go on: the interface, the registration point for a new experiment, and the conventions it must follow should be findable and unambiguous from the code and documentation as committed.

2 Experiment A — Pendulum

- Simulated from the real equation of motion ($\theta'' = -(g/L) \cdot \sin \theta - \text{damping}$), not small-angle shortcut, not a canned animation.
- Parameters: length, gravity, initial angle, damping. Graph: angle vs. time.
- A 'Measure period' feature that computes the period from simulation data and displays it next to the theoretical $2\pi\sqrt{L/g}$, with % difference.

3 Experiment B — Projectile motion

- Parameters: launch angle, initial speed, gravity, optional air-resistance coefficient.
- Before launch: show the **predicted analytic trajectory** (no drag) as a dotted curve.
- On launch: simulate actual flight (with drag if enabled), mark the landing point, and display predicted vs. actual range.

4 Experiment C — Spring-mass oscillator

- Parameters: spring constant k , mass, damping, initial displacement. Visible spring that stretches/compresses.
- Graph: displacement vs. time. Display measured vs. theoretical frequency ($\sqrt{k/m}/2\pi$) for the undamped case.

5 Numerical integration

- Choose your integrator deliberately (semi-implicit Euler, RK4, ...) and **justify the choice in the README**, including why plain explicit Euler is problematic for oscillators.
- Physics functions must be separated from rendering so they are testable without a browser canvas.

Bonus (only if core is complete)

- Comparison mode: run two parameter sets of the same experiment side by side.
- Unit tests on the physics functions (e.g. Vitest).

- Energy plot (kinetic/potential/total) demonstrating conservation quality of your integrator.

Constraints & rules

- **Timebox: 2–3 days** of effort. Submit what you have at the deadline; scoping decisions are part of the evaluation.
- **Three.js is required** for all 3D rendering. Physics/math must be your own or AI-assisted code — **no physics engines** (no cannon.js, rapier, etc.) unless the brief says otherwise.
- Keep third-party dependencies minimal and justify each one in the README.
- **Target:** recent Chrome/Firefox on desktop. Mobile support is not required.

Deliverables

- **A Git repository** with a clear, incremental commit history. **A single giant commit is not acceptable.**
- **README.md** covering: how to run the project, architecture overview and key decisions, known limitations, and what you would do next with more time.
- **AI_USAGE.md** documenting your AI workflow: which tools and models you used (Claude, Copilot, ChatGPT, etc.), 3–5 representative prompts, at least **two concrete cases where the AI output was wrong or suboptimal** and how you detected and fixed it, which parts you wrote fully by hand, and an answer to this question: **what did you set up before writing any code, and why?**
- **Your agent environment**, committed to the repository. This is mandatory, not optional. Commit the persistent configuration you gave your AI tooling into a directory in the repo (for example `.claude/`, `.agents/`, `.cursor/`, `.github/` — these are examples of where such configuration commonly lives, not requirements): rules, instructions, workflows, subagent definitions, MCP configuration, or whatever reusable components you found useful. Explain in the README or in that directory what each piece was for, including at least one rule you added *because* the AI kept getting something wrong. Commit it as you go, not at the end — we read the git history, and configuration that grew alongside the code reads very differently from configuration written in the final hour.
- **The app must run** with a simple, documented command (e.g. `npm install && npm run dev`). Vite + vanilla JS/TS or any framework is fine; Three.js must be the 3D engine.

Review session

After submission there will be a ~30 minute call. You will walk us through the architecture and then make a small live modification we choose on the spot. Be sure you deeply understand every part of your codebase, including AI-generated parts.

A note on AI tools

You are **encouraged** to use AI assistants — this assessment explicitly measures how well you work with them. We are not testing whether you can avoid AI; we are testing whether you can direct it, verify it, and keep the codebase coherent. Blindly pasted AI output is easy to spot in the review session and will count against you.

Evaluation rubric

CRITERION	WEIGHT	WHAT WE LOOK FOR
Functionality & completeness	20%	All three experiments run correctly on the shared framework.
Architecture & code quality	30%	Genuine shared abstractions: schema-driven UI, common interface, sim/render separation, no copy-paste triplication.
Physics & numerics correctness	20%	Frame-rate-independent results, sound integrator choice, measured values match theory within tolerance.
AI collaboration quality	10%	AI_USAGE.md shows you verified AI's math and unified its inconsistent output instead of accepting three divergent implementations.
AI-extensibility	10%	A committed agent environment that is real, specific and clearly earned during the work. And: an unseen experiment drops into your platform as roughly one new file plus a registry entry, with documentation that lets an agent find the framework contract quickly and without guessing.
Live modification session	10%	You can add a small feature live (e.g. a new parameter or measurement) using your own framework.

Good luck — we look forward to seeing your lab!